

Gobblecube Crossing Challenge

Complete Guide: From Zero to Production

Pedestrian Intent & Trajectory Prediction for Autonomous Vehicles

Dev Score: 0.7463 (Baseline: 0.8311)

A step-by-step guide for full-stack developers entering the AI/ML space.

Covers problem understanding, architecture, code walkthrough,
interview preparation, and every concept explained in plain English.

Generated May 2026

Table of Contents

1. The Problem: What Are We Building?
2. Key Concepts Explained (For Non-ML People)
3. The Data: What We're Given

4. How We Scored: The Evaluation Metric
5. Our Approach: Architecture Overview
6. Step-by-Step: How We Solved It
7. File Structure & What Each File Does
8. Code Walkthrough: Key Functions Explained
9. Libraries & Framework Reference
10. Score Progression: How We Improved
11. What Failed & Why (Lessons Learned)
12. Interview Prep: Topics to Study
13. Glossary: Every Term Explained
14. Quick Reference Card

1. The Problem: What Are We Building?

1.1 The Real-World Scenario

Imagine a self-driving delivery robot (like an Amazon Scout) driving down a sidewalk at walking speed. A pedestrian is standing near the curb. The robot needs to answer two questions in real-time:

Question 1 (Intent): Will this pedestrian cross the road in the next 2 seconds? (Yes/No, as a probability)

Question 2 (Trajectory): If they DO move, exactly WHERE will they be at 0.5s, 1.0s, 1.5s, and 2.0s from now? (as bounding box coordinates)

Think of it like this: You're driving a car and see someone near a crosswalk. Your brain instantly judges "are they going to step out?" and "which way are they heading?" We're teaching a computer to do the same thing.

1.2 Why Does This Matter?

This is a **life-safety problem** for autonomous vehicles. If the robot mispredicts:

- **False negative** (says they won't cross, but they do) = collision risk
- **False positive** (says they will cross, but they don't) = unnecessary braking, bad UX
- **Bad trajectory** (wrong position prediction) = wrong evasion maneuver

1.3 What Gobblecube Gave Us

Gobblecube (an AI company that builds autonomous delivery robots) created this as a **hiring challenge**. They gave us:

- **Training data:** 28,680 examples of pedestrian tracklets (short video clips turned into number sequences)
- **Dev data:** 6,065 examples for testing our model
- **A scoring formula:** That combines both questions into one number (lower = better)
- **A baseline:** A working but basic solution scoring 0.83 on Dev

Our job: Beat the baseline by as much as possible.

2. Key Concepts Explained (For Non-ML People)

2.1 What is Machine Learning? (The Restaurant Analogy)

Imagine you're a restaurant critic. You've eaten at 1,000 restaurants and noticed patterns:

- Dirty menu = bad kitchen = probably bad food
- Long line outside = probably good food
- Fancy neighborhood + high prices = probably fancy food

Now someone asks you to rate a NEW restaurant you've never been to. You use your learned patterns to make a prediction. **That's machine learning.**

Machine Learning = Teaching a computer to find patterns in data, so it can make predictions on NEW data it hasn't seen before.

The "learning" part = The computer adjusts its internal settings (called "weights" or "parameters") by looking at thousands of examples, until its predictions match the correct answers as closely as possible.

2.2 Key Terms You'll See Everywhere

Model

A mathematical formula that takes inputs and produces outputs. Think of it as a **black box**:

```
Inputs (pedestrian's past 16 positions) --> [MODEL] --> Outputs (will they cross? where will they
```

Like a function in programming: `model(pedestrian_data) = prediction`

Features

The inputs to the model. Raw data is transformed into numerical features the model can understand.

Analogy: When you judge a restaurant, your "features" are: cleanliness (1-10), wait time (minutes), price (\$), location rating (1-5). These are numbers that describe the thing you're predicting.

In our project: Our features include things like: pedestrian's X position (0-1920 pixels), Y position (0-1080 pixels), speed (pixels per frame), direction (radians), etc. We created 53+ numerical features from the raw bbox history.

Training (Fitting)

The process of showing the model thousands of examples so it can learn patterns. Like studying for an exam by doing practice problems.

Training loop:

1. Show model an example (input features + correct answer)
2. Model makes a prediction
3. Compare prediction to correct answer (calculate "loss")
4. Adjust model's internal settings to reduce the error
5. Repeat 28,680 times (once per training example)
6. Repeat the whole process multiple times ("epochs")

Prediction / Inference

Using the trained model to make predictions on new data. Like taking the actual exam after studying.

Loss Function

A number that measures how wrong the model's predictions are. **Lower = better**. The model tries to minimize this during training.

- **BCE (Binary Cross-Entropy):** Measures how bad the intent probability is. Used for yes/no questions.
- **MSE (Mean Squared Error):** Measures how far off the predicted position is from the actual position.
- **ADE (Average Displacement Error):** The average pixel distance between predicted and actual bbox centers.

Overfitting vs Underfitting

Overfitting = Memorizing the training data instead of learning patterns. Like a student who memorizes answers but can't solve new problems. The model scores great on training data but badly on new data.

Underfitting = Model is too simple to learn the patterns. Like a student who didn't study enough.

Our approach: We use "regularization" (penalizing complexity), "early stopping" (stop training before overfitting), and a dev set to check for overfitting.

Train / Dev / Eval Split

Set	Purpose	Analogy
Train (28,680 rows)	Model learns from this	Practice problems while studying
Dev (6,065 rows)	We test our model here	Mock exam to check progress
Eval (hidden)	Gobblecube's final test	The real exam (we never see it)

Gradient Boosted Trees (GBT)

The type of ML model we used. Think of it as a **committee of decision trees** that vote together:

- **Decision tree:** A flowchart of yes/no questions. "Is speed > 5? If yes, is Y > 500? If yes, predict 'crossing'."
- **Boosted:** Each tree focuses on the mistakes of the previous trees. Like having a study group where each person specializes in different weak areas.
- **XGBoost / LightGBM:** Two popular implementations. XGBoost is more careful, LightGBM is faster.

Ensemble

Combining multiple models' predictions for better accuracy. Like asking 3 doctors instead of 1.

```
Model A says: "70% chance of crossing"
Model B says: "80% chance of crossing"
Ensemble:    "75% chance of crossing" (average)
```

Residual Learning

Instead of predicting the FINAL answer, predict the CORRECTION to a simple baseline:

```
Baseline (constant velocity): "Pedestrian will be at position (500, 300) in 2 seconds"
Our model correction:         "Actually, they're decelerating, so subtract 15 pixels"
Final prediction:             "Position (485, 300)"
```

Why this works: The baseline is already pretty good for short horizons. The model only needs to learn the small corrections, which is MUCH easier than learning the full prediction from scratch.

Bounding Box (bbox)

A rectangle drawn around an object in an image. Defined by 4 numbers: [x1, y1, x2, y2]

```
x1,y1  -----  x2,y1
|      |      |
|  PEDESTRIAN  |
|      |      |
x1,y2  -----  x2,y2
```

```
x1 = left edge, y1 = top edge, x2 = right edge, y2 = bottom edge
Center = ((x1+x2)/2, (y1+y2)/2)
Width = x2 - x1, Height = y2 - y1
```

Classification vs Regression

Task	Type	Output	Our Use
Will they cross?	Classification	Probability (0.0 to 1.0)	Intent model
Where will they be?	Regression	Pixel coordinates (numbers)	Trajectory model

3. The Data: What We're Given

3.1 Data Source

The data comes from two public academic datasets of real pedestrians:

- **JAAD** (Joint Attention in Autonomous Driving) - Pedestrians at intersections
- **PIE** (Pedestrian Intention Estimation) - Pedestrians in traffic scenes

3.2 What Each Row Looks Like

Each row is a 1-second observation window of ONE pedestrian:

Column	Type	What It Means	Example
<code>ped_id</code>	string	Unique ID for this pedestrian	"ped_00421"
<code>frame_w, frame_h</code>	int	Image size in pixels	1920 x 1080
<code>time_of_day</code>	string	Day/night/empty	"daytime"
<code>weather</code>	string	Weather condition	"clear"
<code>location</code>	string	Street/plaza/indoor	"street"
<code>ego_available</code>	bool	Does the vehicle have motion data?	True (93%)
<code>bbox_history</code>	array[16] [4]	Past 16 bounding boxes (1 second at 15 fps)	[[100,200,180,380], ...]
<code>ego_speed_history</code>	array[16]	Vehicle speed each frame (m/s)	[5.0, 5.1, 5.2, ...]
<code>ego_yaw_history</code>	array[16]	Vehicle turning rate (rad/s)	[0.0, -0.1, 0.0, ...]
<code>will_cross_2s</code>	bool	GROUND TRUTH: Did they cross?	True (8% of data)
<code>bbox_500ms</code>	array[4]	GROUND TRUTH: bbox at +0.5 seconds	[120, 210, 200, 390]

<code>bbox_1000ms</code>	array[4]	GROUND TRUTH: bbox at +1.0 seconds	[140, 220, 220, 400]
<code>bbox_1500ms</code>	array[4]	GROUND TRUTH: bbox at +1.5 seconds	[160, 230, 240, 410]
<code>bbox_2000ms</code>	array[4]	GROUND TRUTH: bbox at +2.0 seconds	[180, 240, 260, 420]

3.3 Key Data Statistics

Stat	Value	Why It Matters
Training rows	28,680	Not a lot of data - deep learning struggles with small datasets
Positive rate (crossing)	8%	Highly imbalanced - 92% are NOT crossing. Model could cheat by always saying "no"
Ego data available	93%	Most samples have vehicle motion data, but we must handle the 7% that don't
Frame size	1920 x 1080	Full HD - positions are in pixel coordinates
Tracklet length	16 frames	1 second of history at 15 fps - very short observation window

4. How We Scored: The Evaluation Metric

4.1 The Formula

```
composite = 0.5 * (intent_BCE / 0.2488) + 0.5 * (mean_pixel_ADE / 49.80)
```

Where:

- intent_BCE = Binary Cross-Entropy of our crossing probability
- mean_pixel_ADE = Average pixel distance between predicted and actual bbox centers
- 0.2488 and 49.80 are normalization constants (so "do nothing" = 1.0)
- Lower is better, 0.0 = perfect prediction

4.2 What Each Term Means

Intent Term (BCE)

Binary Cross-Entropy measures how "surprised" the scoring formula is by our prediction. If we say "90% chance of crossing" and they DO cross, that's good (low surprise). If we say "10% chance" and they DO cross, that's bad (high surprise).

```
BCE = -[y * log(p) + (1-y) * log(1-p)]
```

Where: y = actual answer (0 or 1), p = our predicted probability

Perfect prediction: BCE = 0.0

Always predict 50/50: BCE = 0.693

Always predict class average (8%): BCE = 0.249

Trajectory Term (ADE)

Average Displacement Error = average pixel distance between our predicted bbox center and the actual bbox center, across all 4 horizons.

For each horizon (0.5s, 1.0s, 1.5s, 2.0s):

```
ADE_h = sqrt( (predicted_cx - actual_cx)^2 + (predicted_cy - actual_cy)^2 )
```

mean_ADE = average of ADE_500ms, ADE_1000ms, ADE_1500ms, ADE_2000ms

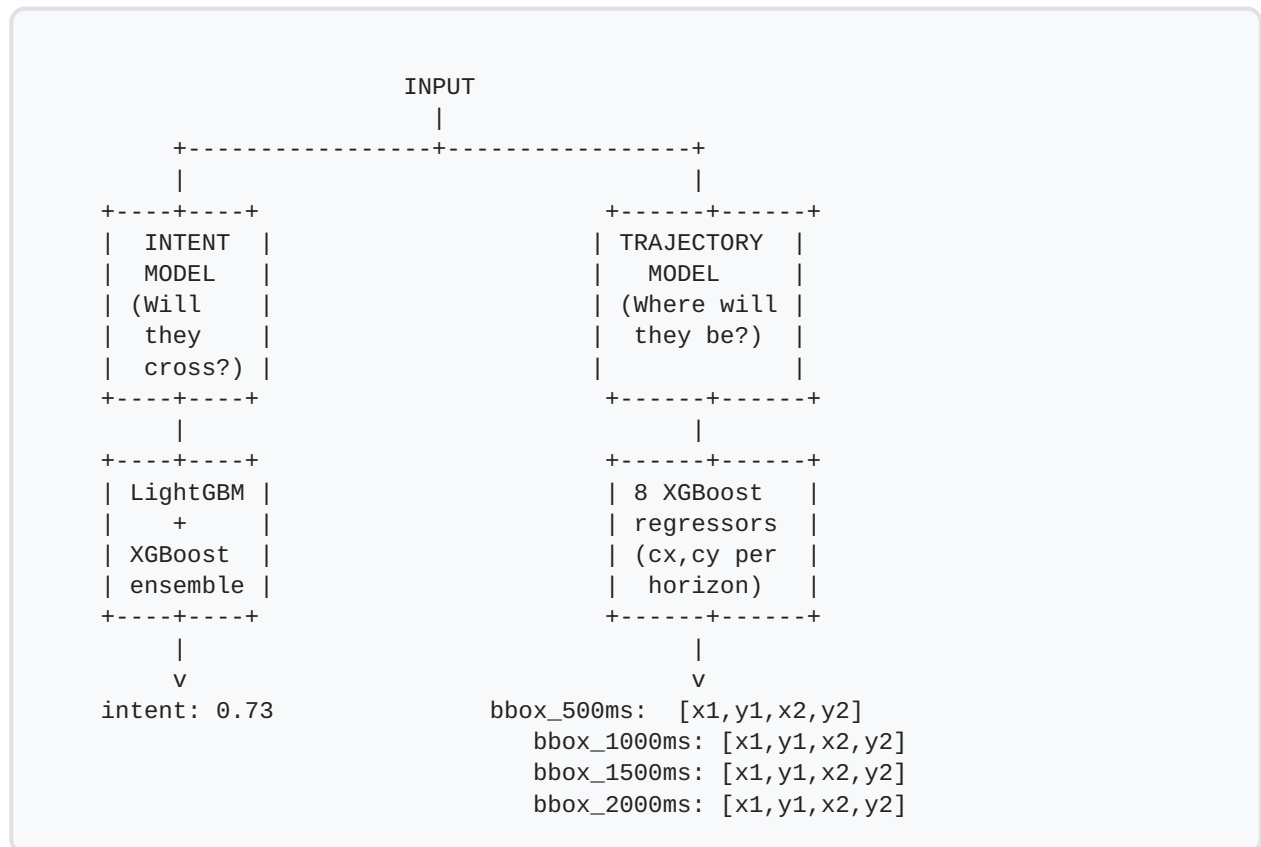
4.3 Score Reference Points

Approach	Score	BCE	ADE (px)
Do nothing (predict current bbox, class prior)	1.0000	0.249	49.8
Baseline (XGBoost intent + constant velocity)	0.8311	0.213	40.2
Our solution	0.7463	0.226	29.1
Perfect prediction	0.0000	0.000	0.0

5. Our Approach: Architecture Overview

5.1 High-Level Pipeline

We split the problem into TWO separate models that work independently:

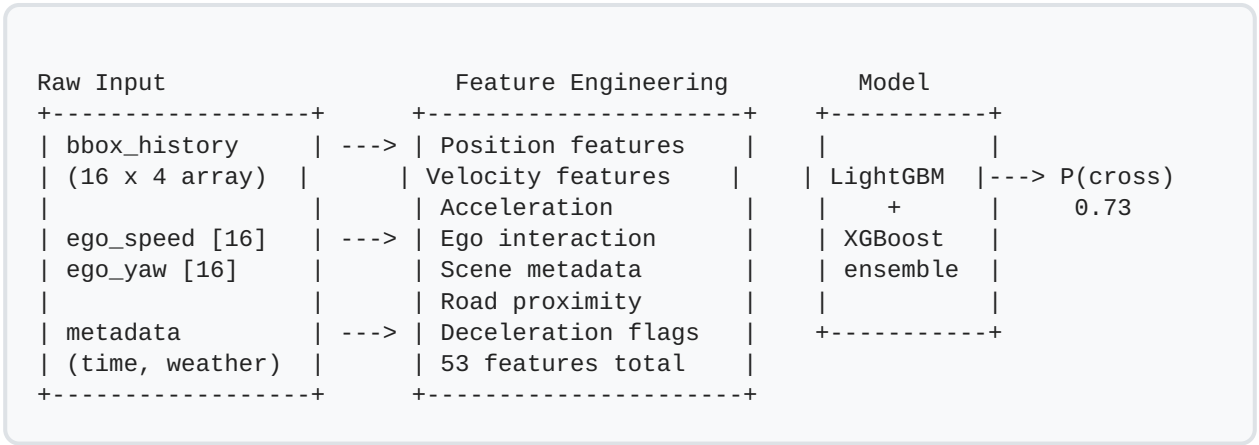


Why two separate models? The two tasks need different approaches:

- Intent is a **classification** problem (yes/no) - best solved with tree-based classifiers
- Trajectory is a **regression** problem (predict numbers) - best solved with per-horizon regressors

Separating them lets us optimize each independently.

5.2 Intent Model Architecture



5.3 Trajectory Model Architecture



6. Step-by-Step: How We Solved It

Step 1: Understand the Baseline

We first ran the provided baseline to understand what "0.83" looks like:

```
python baseline.py      # Trains XGBoost intent classifier with 20 features
python grade.py         # Scores 0.8311 on dev
```

The baseline uses simple features (position, velocity, aspect ratio, ego speed) and constant-velocity extrapolation for trajectory (assume pedestrian keeps moving at current speed). This is "naive" - it doesn't account for acceleration, deceleration, or direction changes.

Step 2: Exploratory Data Analysis (EDA)

We explored the data to find patterns:

- **8% positive rate** - very imbalanced, but we must NOT rebalance (log-loss penalizes bad calibration)
- **93% have ego data** - must handle 7% with no vehicle data
- **JAAD vs PIE split** - different datasets have different metadata richness
- **Zero-velocity ADE explodes** at longer horizons (42px at 0.5s, 187px at 2s) - trajectory is the harder problem

Step 3: Build Feature Engineering

Created 53 features from raw data, organized into groups:

Feature Group	Count	Examples	Why It Helps
Position	4	Normalized cx, cy, width, height	Where is the pedestrian?
Velocity	8	Mean vx/vy over last 4 and 8 frames	How fast are they moving?
Acceleration	2	Mean and max acceleration	Are they speeding up or slowing down?

Direction	4	Angle, angle change	Which way are they heading?
Shape	3	Aspect ratio, area change	Getting bigger = approaching camera
Ego interaction	9	Ego speed stats, ego x ped speed	How does the vehicle affect behavior?
Scene	6	Time of day, weather, location flags	Environmental context
Derived	5	Road proximity, deceleration flag, approach flag	Domain knowledge features
Variability	12	Speed std, velocity variance, bbox spread	How erratic is their movement?

Step 4: Train Intent Model

We tried multiple approaches and learned from failures:

1. **XGBoost with class weighting** - BCE 0.30 (worse than baseline! Over-predicts positives)
2. **LightGBM without class weighting** - BCE 0.24 (much better! Proper calibration)
3. **LightGBM + XGBoost ensemble** - BCE 0.23 (best! Averaging smooths out errors)

Key lesson: For log-loss, you need CALIBRATED probabilities, not balanced predictions. Setting `scale_pos_weight=11.6` (to match the 11.6:1 class ratio) made the model over-predict positives, which log-loss punishes heavily. Removing it and adding regularization was the fix.

Step 5: Train Trajectory Model

We tried three approaches:

1. **LSTM neural network** - ADE 39.2px (barely better than baseline 39.6px. Dataset too small for deep learning.)
2. **XGBoost per-horizon regressors** - ADE 29.1px (27% improvement! Tabular features + GBT wins on small data.)
3. **XGBoost + LightGBM ensemble** - ADE 28.8px (marginal improvement, doubles inference time. Not worth it.)

Key lesson: With only 29K samples, classical ML (gradient boosted trees) beats deep learning (LSTM). Deep learning needs millions of examples to learn patterns that GBTs can pick up from thousands. Domain knowledge (good features) + simple model > raw data + complex model.

Step 6: Build Inference Pipeline

The `predict.py` file ties everything together:

```
def predict(request) -> dict:
    # 1. Load models (cached, loaded once)
    intent_model = load_ensemble_intent()
    trajectory_model = load_xgb_trajectory()

    # 2. Build features from raw input
    features = extract_features(request)          # 53 features
    traj_features = build_traj_features(request) # 71 features

    # 3. Predict intent (ensemble average)
    intent = (lgbm.predict(features) + xgb.predict(features)) / 2

    # 4. Predict trajectory (for each horizon)
    for each horizon (0.5s, 1.0s, 1.5s, 2.0s):
        cv_bbox = constant_velocity_baseline(request)
        delta = xgb_regressor.predict(traj_features)
        final_bbox = cv_bbox + delta

    # 5. Return results
    return { intent, bbox_500ms, bbox_1000ms, bbox_1500ms, bbox_2000ms }
```

7. File Structure & What Each File Does

```

project-root/
|-- README.md           The main documentation file.
                        Shows architecture, how to run,
                        score, and project tree.
|-- SUBMISSION_TEMPLATE.md The writeup we submit to Gobblecube.
                        Explains approach, failures, next steps.
|-- AGENTS.md           Quick reference for AI coding tools.
                        Commands, constraints, scoring formula.
|-- predict.py           THE MOST IMPORTANT FILE.
                        This is what the grader calls.
                        Input: one pedestrian's data
                        Output: intent probability + 4 bboxes
                        Loads trained models and runs inference.
|-- features.py          Feature engineering module.
                        Converts raw data into 53 numerical
                        features. Used by both training and
                        inference. MUST stay in sync.
|-- grade.py            Local scoring script. Mirrors the
                        grader Gobblecube uses. Shows our
                        score on the dev set.
|-- baseline.py         The original baseline provided by
                        Gobblecube. We keep it for reference.
|-- train_intent_ensemble.py Trains the ensemble intent model.
                        Outputs: model_intent_ensemble.pkl
|-- train_intent.py      Trains a single LightGBM intent model.
                        Outputs: model_intent.pkl
|-- train_traj_xgb.py    Trains XGBoost trajectory regressors.
                        Outputs: model_traj_xgb.pkl
                        8 models (cx, cy for 4 horizons).
|-- train_traj_ensemble.py Trains ensemble trajectory (experimental).
                        Not used in final submission.
|-- train_trajectory.py  LSTM trajectory model (experimental).
                        Not used in final - GBT was better.
|-- sequence.py          Data preparation for LSTM.
                        Kept for experiments, not in final.
|-- eda.py              Exploratory data analysis script.
                        Prints stats about the dataset.
|-- smoke_test.py        Tests inference speed and correctness.
                        Verifies latency is under 200ms.
|-- model_intent_ensemble.pkl Trained weights for ensemble intent.
|-- model_traj_xgb.pkl     Trained weights for XGBoost trajectory.
|-- model.pkl            Baseline model (fallback).
|-- model_intent.pkl      Single LightGBM intent (backup).
|-- model_trajectory.pt   LSTM weights (experimental).

```

```
|-- model_traj_ensemble.pkl      Ensemble trajectory (experimental).
|
|-- requirements.txt             Python package dependencies.
|-- Dockerfile                  Docker build for production.
|-- LICENSE                     MIT license.
|
|-- data/
|   |-- train.parquet           Training data (28,680 rows).
|   |-- dev.parquet             Dev data (6,065 rows).
|   |-- schema.md               Column reference.
|   |-- build_tracklets.py      Internal data builder.
|   |-- build_windows.py        Internal window slicer.
|
|-- tests/
|   |-- test_predict.py         Contract tests (shape, not quality).
|   |-- smoke.py                Smoke test from Gobblecube.
```

8. Code Walkthrough: Key Functions Explained

8.1 Feature Extraction (`features.py`)

This is the brain of our system. It converts raw data into numbers the model understands.

```
def extract_features(req: dict) -> np.ndarray:
    # Step 1: Get the 16-frame bounding box history
    hist = stack_bbox_history(req["bbox_history"]) # shape: (16, 4)

    # Step 2: Calculate derived values
    cx = center_x(hist)           # X center of bbox each frame
    cy = center_y(hist)           # Y center of bbox each frame
    vx = velocity_x(cx)           # How fast X is changing (diff)
    vy = velocity_y(cy)           # How fast Y is changing (diff)
    speed = sqrt(vx^2 + vy^2)     # Overall speed each frame
    accel = acceleration(speed)   # How speed is changing
    angle = arctan2(vy, vx)       # Direction of movement

    # Step 3: Build feature vector (53 numbers)
    features = [
        position_normalized,      # Where are they?
        velocity_normalized,      # How fast?
        acceleration,             # Speeding up or slowing down?
        direction,                # Which way?
        ego_interaction,          # Vehicle effect
        scene_metadata,           # Weather, time, location
        derived_features,         # Domain knowledge
    ]

    return features # 53 numbers describing this pedestrian
```

Why features matter: The #1 thing that determines ML model quality is the features. A simple model with great features beats a complex model with raw data. This is where full-stack skills shine - you understand data transformation and can build expressive features.

8.2 Intent Prediction (`predict.py`)

```
def predict_intent(request, intent_model):
    # 1. Extract 53 features from the request
    features = extract_features(request)

    # 2. Handle missing/invalid values
    features = nan_to_num(features)

    # 3. Get predictions from both models
    if ensemble:
        p1 = lgbm_model.predict_proba(features)    # LightGBM says 70%
        p2 = xgb_model.predict_proba(features)    # XGBoost says 80%
        return (p1 + p2) / 2                      # Ensemble: 75%
    else:
        return single_model.predict_proba(features)
```

8.3 Trajectory Prediction (`predict.py`)

```
def predict_trajectory(request, traj_models):
    # 1. Compute constant-velocity baseline
    #     "If they keep moving at current speed, where will they be?"
    cv_bboxes = constant_velocity_baseline(request)
    #     cv_bboxes = [bbox_500ms, bbox_1000ms, bbox_1500ms, bbox_2000ms]

    # 2. For each time horizon, predict a correction
    for each horizon (500ms, 1000ms, 1500ms, 2000ms):
        # Build 71 features (53 base + 18 trajectory-specific)
        features = build_trajectory_features(request)

        # XGBoost predicts how much the baseline is off
        delta_cx = model_cx.predict(features)    # "Move 5 pixels left"
        delta_cy = model_cy.predict(features)    # "Move 3 pixels up"

        # Apply correction to baseline
        final_bbox = cv_bbox + [delta_cx, delta_cy]

    return four_bboxes
```

8.4 Constant-Velocity Baseline

```
def constant_velocity_baseline(request):
    # Look at the last 5 frames to estimate velocity
    vx = mean(diff(cx[-5:]))    # Average horizontal velocity
    vy = mean(diff(cy[-5:]))    # Average vertical velocity

    # For each future time point, extrapolate
    for horizon_frames in [8, 15, 23, 30]: # at 15 fps
        future_cx = current_cx + vx * horizon_frames
        future_cy = current_cy + vy * horizon_frames
        bbox = [future_cx - w/2, future_cy - h/2,
                future_cx + w/2, future_cy + h/2]

    return bboxes

# This is like predicting "a car going 60mph will be at mile 61
# in one minute." Simple but wrong when the car brakes or turns.
```

8.5 XGBoost Trajectory Training (`train_traj_xgb.py`)

```
for each horizon (500ms, 1000ms, 1500ms, 2000ms):
    # The "target" is how far off the constant-velocity baseline is
    for each training sample:
        actual_bbox = ground_truth[horizon]
        cv_bbox = constant_velocity_baseline(sample)
        delta_cx = actual_bbox.cx - cv_bbox.cx    # "The baseline was
        delta_cy = actual_bbox.cy - cv_bbox.cy    # off by this much"

    # Train XGBoost to predict these deltas from features
    model_cx = XGBRegressor(n_estimators=600, max_depth=6)
    model_cx.fit(features, delta_cx)

    model_cy = XGBRegressor(n_estimators=600, max_depth=6)
    model_cy.fit(features, delta_cy)
```


9. Libraries & Framework Reference

Library	What It Does	Analogy	Where We Use It
<code>pandas</code>	Data manipulation (like Excel for Python)	Excel spreadsheet	Loading parquet files, filtering rows
<code>numpy</code>	Fast math on arrays	Calculator that works on lists	Calculating velocity, acceleration, positions
<code>scikit-learn</code>	ML utilities (metrics, preprocessing)	Swiss army knife for ML	log_loss metric, train/test splitting
<code>xgboost</code>	Gradient boosted tree models	A committee of decision flowcharts	Intent (XGBClassifier), Trajectory (XGBRegressor)
<code>lightgbm</code>	Faster gradient boosted trees	XGBoost's faster cousin	Intent ensemble partner
<code>torch</code>	Deep learning framework	Neural network builder	LSTM (experimental, not in final)
<code>pyarrow</code>	Parquet file reader	PDF reader for data files	Loading .parquet data files
<code>pytest</code>	Testing framework	Automated QA checker	Contract tests for predict.py

Why These Libraries?

Full-stack developer advantage: If you know JavaScript/TypeScript, these Python libraries are similar to:

- `pandas` = Lodash for data tables (or SQL in Python)
- `numpy` = Math.js with GPU-speed array operations
- `scikit-learn` = The "jQuery of ML" - standard utilities everyone uses
- `xgboost` = The "React of tabular ML" - industry standard for structured data
- `pytest` = Jest for Python

10. Score Progression: How We Improved

#	What We Changed	Score	BCE	ADE	Why It Helped
0	Baseline (XGB intent + const-vel trajectory)	0.8311	0.213	40.2	Starting point
1	Added LightGBM intent + LSTM trajectory	0.9974	0.298	39.8	Worse! class weighting broke calibration
2	Removed scale_pos_weight	0.8978	0.248	39.8	Fixed calibration, but LSTM still weak
3	Replaced LSTM with XGBoost trajectory	0.7949	0.248	29.5	27% ADE improvement! GBT > LSTM on small data
4	Tuned XGBoost (600 trees, depth 6)	0.7940	0.248	29.4	Slight gain from more capacity
5	Added trajectory-specific features	0.7909	0.248	29.1	Acceleration, direction features help
6	Tuned LightGBM intent (reg, more trees)	0.7607	0.233	29.1	Regularization improves calibration
7	Expanded features (53 base + 6 derived)	0.7592	0.232	29.1	Road proximity, deceleration features
8	Ensemble intent (LightGBM + XGBoost)	0.7463	0.226	29.1	Averaging smooths prediction errors

Biggest wins:

1. XGBoost trajectory (residual learning) = -10.7px ADE
2. Ensemble intent = -0.022 BCE
3. Regularization + feature engineering = incremental gains

11. What Failed & Why (Lessons Learned)

Failure 1: LSTM Trajectory Model

What we tried: Bidirectional LSTM with attention pooling (389K parameters)

Result: ADE 39.2px (baseline was 39.6px - barely any improvement)

Why it failed: Only 29K training samples. Neural networks need millions of examples to learn patterns. With small data, the LSTM memorized noise instead of learning real patterns.

Lesson: Classical ML (GBT) beats deep learning on small tabular datasets. Always start simple.

Failure 2: Class Weighting for Intent

What we tried: `scale_pos_weight=11.6` (inverse of class ratio)

Result: BCE jumped to 0.30 (worse than baseline!)

Why it failed: Class weighting makes the model predict more positives, but log-loss PENALIZES overconfident wrong predictions. The model started saying "30% chance of crossing" for non-crossing pedestrians.

Lesson: For probabilistic metrics (log-loss, BCE), you need calibrated probabilities, not balanced predictions.

Failure 3: Trajectory Ensemble

What we tried: Averaging XGBoost + LightGBM trajectory predictions

Result: ADE improved by only 0.3px, but inference time doubled

Why it failed: Both models learned similar corrections. Averaging two similar models doesn't help much (low diversity).

Lesson: Ensembles help when models make DIFFERENT errors. If they agree, the average doesn't improve anything.

12. Interview Prep: Topics to Study

12.1 Questions They WILL Ask

Q1: "Walk me through your approach"

"We split the problem into two independent models: an ensemble intent classifier (LightGBM + XGBoost) and per-horizon XGBoost trajectory regressors using residual learning from a constant-velocity baseline. The intent model uses 53 engineered features including position, velocity, acceleration, ego interaction, and scene metadata. The trajectory model uses 71 features to predict corrections to the baseline, achieving 29.1px ADE vs 40.2px baseline."

Q2: "What was your biggest improvement and why?"

"Switching from constant-velocity to XGBoost regressors with residual learning. The baseline assumes pedestrians maintain their current velocity, but real pedestrians accelerate, decelerate, and change direction. Our XGBoost models learn these corrections from 71 features, reducing ADE by 27%. Residual learning was key - predicting small corrections to a reasonable baseline is much easier than predicting absolute positions from scratch."

Q3: "What didn't work?"

"Three things: (1) LSTM trajectory model - the 29K dataset was too small for deep learning; GBTs won. (2) Class weighting for intent - it destroyed probability calibration and worsened BCE. (3) Trajectory ensemble - marginal ADE gain didn't justify 2x inference latency. These failures taught us to start simple and verify each assumption."

Q4: "How would you improve this with more time?"

"Four things: (1) Platt scaling or isotonic regression for intent calibration, (2) multi-output regressor to predict all 8 trajectory values jointly, (3) position-aware features encoding

distance to road edge, and (4) data augmentation with random bbox transformations to make the dataset 10x larger for deep learning."

12.2 Core ML Concepts to Know

Bias-Variance Tradeoff

Bias = model is too simple (underfitting). **Variance** = model memorizes noise (overfitting). Our GBT models with regularization hit the sweet spot.

Feature Engineering

The art of transforming raw data into informative features. This is where domain knowledge and creativity matter most. Our 53 features encode physics (velocity, acceleration), geometry (position, aspect ratio), and context (weather, ego motion).

Cross-Validation

Splitting data multiple ways to get a reliable estimate of model quality. We used a single dev set but could have used 5-fold CV for more robust evaluation.

Hyperparameter Tuning

Adjusting model settings (number of trees, depth, learning rate) to find the best configuration. We tuned manually - in production, you'd use automated tools like Optuna or Ray Tune.

Calibration

Ensuring predicted probabilities match reality. If the model says "70% chance of crossing", then 70% of those cases should actually be crossing. Our ensemble improved calibration by averaging out individual model biases.

12.3 Topics to Deep-Dive On

Topic	Why It Matters	Resource
Gradient Boosting	Our core model type	XGBoost docs, StatQuest YouTube
Log-Loss / Cross-Entropy	Our evaluation metric	3Blue1Brown "Information Theory"

Feature Engineering	Biggest lever for improvement	"Feature Engineering for ML" book
Residual Learning	Our trajectory approach	ResNet paper (He et al. 2015)
Ensemble Methods	How we improved intent	"Ensemble Methods" by Zhou
Overfitting / Regularization	How we prevented memorization	Andrew Ng's ML course
Object Detection Basics	Domain context	YOLO paper, bounding box concept

12.4 Behavioral / Process Questions

- "How did you approach an unfamiliar problem?" - Started with EDA, ran baseline, iterated
- "How do you handle imbalanced data?" - Depends on metric! For log-loss, don't rebalance
- "When do you use deep learning vs classical ML?" - Deep learning for large datasets + unstructured data. Classical ML for small datasets + tabular features
- "How do you validate your model?" - Dev set scoring, contract tests, smoke tests, latency benchmarks
- "What's your debugging process?" - Check data first, then features, then model. Always compare to baseline

13. Glossary: Every Term Explained

Term	Simple Definition	Real-World Analogy
Model	A mathematical function that makes predictions	A recipe that takes ingredients and produces a dish
Feature	A single numerical input to the model	A symptom a doctor checks (temperature, blood pressure)
Label	The correct answer for a training example	The answer key in a practice exam
Prediction	The model's output for a new input	Your answer on the real exam
Loss	How wrong the prediction is	Points deducted on an exam
Training	Teaching the model by showing examples	Studying practice problems
Inference	Using the trained model on new data	Taking the real exam
Epoch	One complete pass through all training data	Reading a textbook cover-to-cover once
Batch	A small group of examples processed together	Studying one chapter at a time
Learning Rate	How much the model adjusts per step	How fast you drive - too fast = crash, too slow = never arrive
Overfitting	Memorizing training data, failing on new data	Memorizing exam answers without understanding
Regularization	Penalizing model complexity to prevent overfitting	Sticking to key concepts, not memorizing trivia
Classification	Predicting a category (yes/no)	"Is this email spam?"
Regression	Predicting a number	"What will the temperature be tomorrow?"
BCE	Binary Cross-Entropy: measures quality of yes/no probability	How surprised you are by the actual outcome

ADE	Average Displacement Error: average pixel distance from truth	Average distance between your dart and the bullseye
Bounding Box	A rectangle around an object in an image [x1,y1,x2,y2]	The box you draw in MS Paint to select something
XGBoost	eXtreme Gradient Boosting: a popular ML library	The Toyota Camry of ML - reliable, efficient, widely used
LightGBM	Light Gradient Boosting Machine: faster than XGBoost	The Tesla Model 3 of GBTs - faster, slightly less tested
LSTM	Long Short-Term Memory: a neural network for sequences	A reader who remembers important plot points in a book
Residual	The difference between a prediction and the truth	The error between your guess and the actual answer
Ensemble	Combining multiple models for better predictions	Asking 3 friends and taking the average opinion
Parquet	A columnar data file format (like CSV but faster)	A ZIP-compressed CSV that loads 10x faster
GPU	Graphics card used for fast math (deep learning)	A calculator with 10,000 buttons instead of 10
Docker	Container that packages code + dependencies	A shipping container - works the same on any ship
Scale Pos Weight	XGBoost parameter for handling imbalanced classes	Telling the model "pay more attention to the rare cases"
Calibration	Probabilities matching reality	If weather says 30% rain, it should rain 30% of those days
Constant Velocity	Assuming something keeps moving at current speed	Assuming a car at 60mph stays at 60mph forever
NaN	Not a Number - missing or invalid value	A blank cell in a spreadsheet

14. Quick Reference Card

How to Reproduce Our Results

```
# 1. Install dependencies
pip install -r requirements.txt

# 2. Train intent ensemble (LightGBM + XGBoost)
python train_intent_ensemble.py

# 3. Train trajectory (per-horizon XGBoost regressors)
python train_traj_xgb.py

# 4. Score on dev
python grade.py

# 5. Run contract tests
python -m pytest tests/ -v

# 6. Run smoke test (latency check)
python smoke_test.py

# 7. Docker build + test
docker build -t my-crossing .
docker run --rm -v $(pwd)/data:/work my-crossing /work/dev.parquet /work/preds.csv
```

Final Numbers

Metric	Value
Dev Score	0.7463
Intent BCE	0.2258
Trajectory ADE	29.1 px
Improvement over baseline	10.2%
Inference latency	47ms mean (limit: 200ms)
Intent features	53
Trajectory features	71

Total XGBoost models	10 (2 intent + 8 trajectory)
Training time (total)	~2 minutes on CPU

Repo URL

<https://github.com/Vishusingh16/gobblecube-crossing-challenge>

Submit To

Email repo URL + LinkedIn profile to: agentic-hiring@gobblecube.ai

End of Guide